CRM Analytics: architecture

For the FloLabs team. Proposed design, nothing is built yet. 6 November 2026.

> **In short**
>
> Analytics becomes a page of the CRM, not a separate product. It adds new modules to the CRM's existing backend and front end, runs on the same server, and keeps its data in the same database. There is no new server and no second login. It only reads CRM data and does not change how the CRM works.

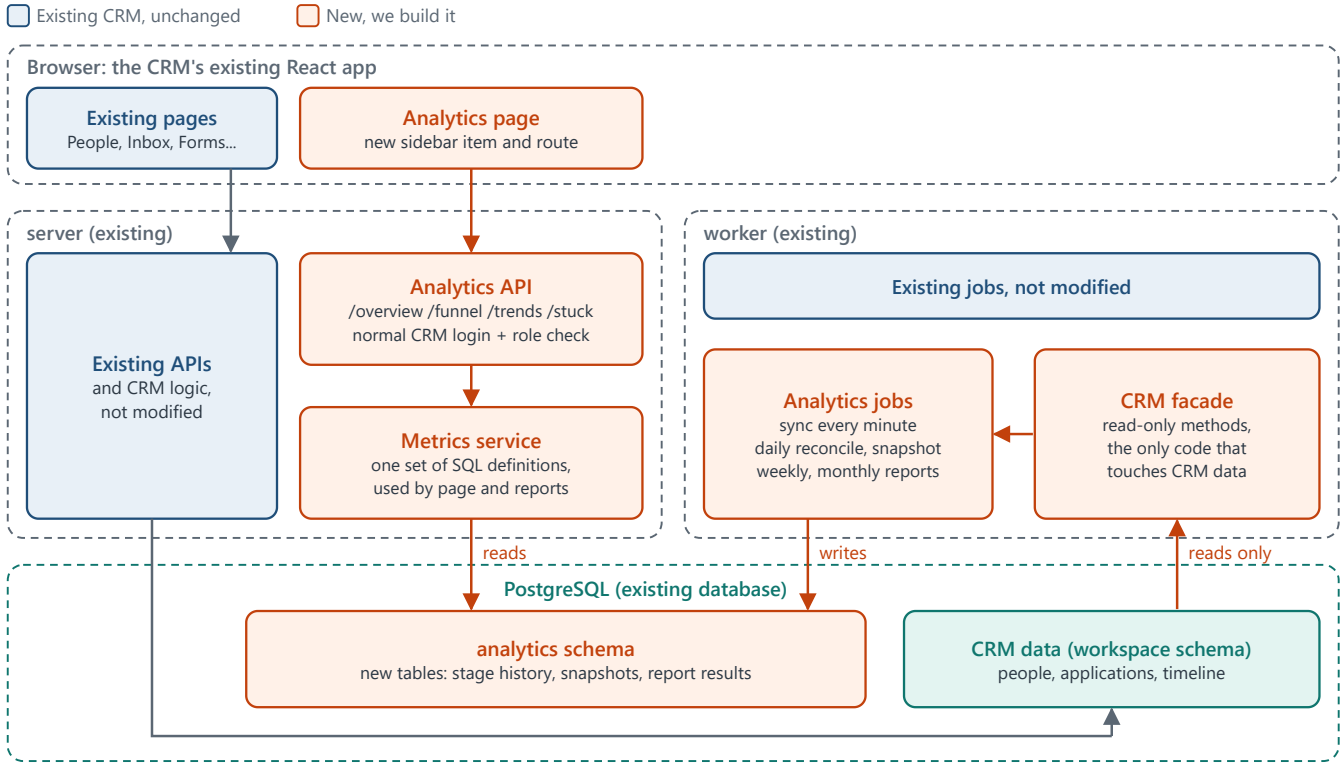
What it gives us

- Pipeline overview, category performance, conversion funnel and historical trends, with filters
- Stage history, conversion, drop-off and time in stage
- Weekly and monthly reports

Key decisions

- **Facade is a module,** so one file knows the CRM's internals
- **Read-only:** the facade has no write methods
- **Heavy work runs in the worker,** and the page reads pre-calculated numbers

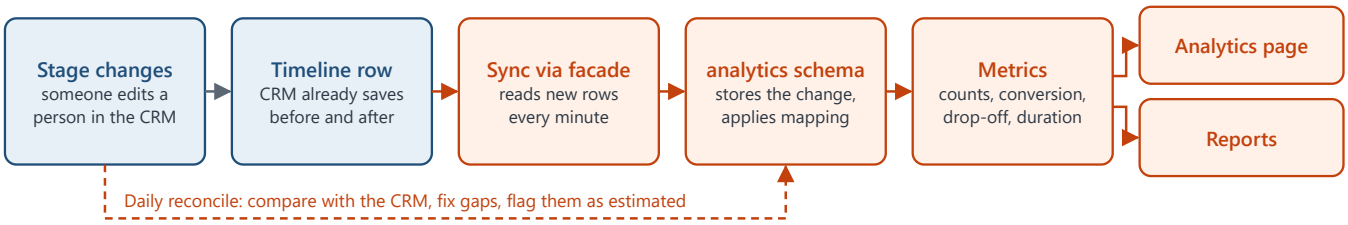
How it fits inside the CRM



Same Docker image, same deploy, same backups. Orange is everything we add. Blue is the CRM as it is today.

How a stage change becomes a chart

We do not hook into the code that saves records. The CRM already records stage changes on its timeline, so a scheduled job reads from there.



One limit: the CRM merges changes made by the same person on the same record within 10 minutes, so A to B to C is stored as A to C. The funnel therefore counts reaching a later stage as having passed the earlier ones.

What we store

Only IDs and categories: no names, emails, files or form answers. This should stay around 150 MB in the first year (estimate).

Table	Holds	Kept for
Stage transitions	Person, from stage, to stage, when	**Forever.** All conversion and duration numbers come from this
Current state	Each person's stage, position, source, dates	Overwritten, can be rebuilt from the CRM
Mapping	Raw stage to lifecycle or reminder, position to program and track	Edited as definitions change
Daily snapshots	Counts per day by stage, track, program, source	Forever, or rolled up later
Report results	Frozen numbers for each weekly and monthly report	Forever, never recalculated
Sync log	Each job run, status, errors	Short term

Why a mapping? The CRM has 25 stages and 39 flat positions, while the forms name 5 tracks, and one program can be spelled three ways. Raw values are stored as they are and mapped when queried, so changing a definition never rewrites history.

Keeping the CRM safe

- New code stays in its own folders
- No hooks in the code that saves records
- Query time limits, so a slow report stops before it hurts the CRM
- A switch turns the whole tool off without a code change
- A second agent reviews every edit to core CRM code
- **Trade-off:** shipping a tool change means a full CRM deploy (about 15 minutes of build, then a restart)

What changes in the code

New folders, separate from existing code

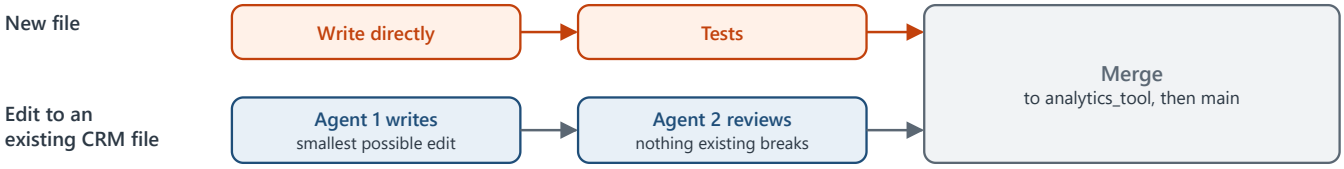
```
twenty-server/src/modules/analytics/
  analytics.module.ts
  facade/      crm-facade.service.ts
  sync/        sync, reconcile, snapshot jobs
  metrics/     metrics.service.ts, SQL
  reports/     report jobs
  api/         analytics controller

twenty-front/src/modules/analytics/
  pages, charts, filters
```

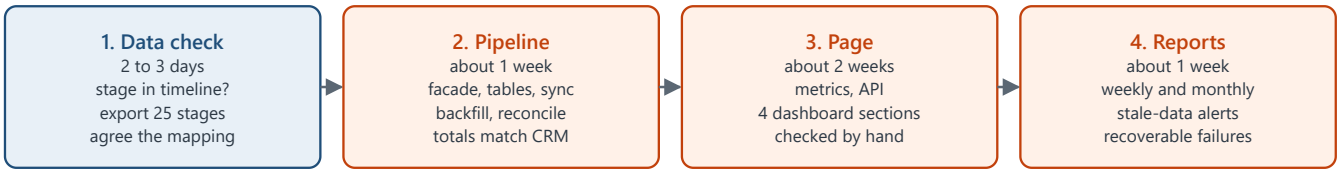
Existing files (a few lines each)

File	Why
`modules.module.ts`	Register the module
`cron-register-all.command.ts`	Schedule the jobs
`AppPath.ts`	Page path
`createWorkspaceRouteObjects.tsx`	Page route
`MainNavigationDrawerScrollableItems.tsx`	Sidebar item

How we change code



Build plan



Durations are rough, for one developer. Work happens on the `analytics_tool` branch, cut from `main`.

What we need to decide or confirm

Question	Proposed	Who
Does the funnel count people or applications?	People, because Stage is on Person. Applications for volume	Team
Which stages are lifecycle or reminder? How do positions group into tracks?	Start from the 5 tracks in the forms	Recruiting
Which stage counts as success?	To be decided	Recruiting
Timezone, drop-off window, stuck thresholds?	Store UTC, the rest as settings	Recruiting
Who can see analytics and receive reports?	Selected roles, fixed internal list	Team
Do stage changes appear in the timeline as expected?	Code says yes, not yet tested on real data. Fallback is a small listener on record events	Dev, phase 1
How are the new tables created?	A separate schema, outside the CRM's migrations. Method to be confirmed	Dev, phase 1